Math 118: Mathematical Methods of Data Theory

Lecture 18: Dynamic Programming

Instructor: Daniel Mckenzie

Dept. of Mathematics, UCLA

February 24, 2020

Discrete Optimization

- We now study **discrete** optimization problems: $\min_{x \in \mathcal{X}} f(x)$ where $\mathcal{X}$ is a **finite set**.
- **Example:** Let $G = (V, E)$ be a graph. We say that $S \subset V$ is *independent* if no edges $\{i, j\}$ with $i, j \in S$. Problem:

$$\max_{\substack{S \subset V \\ S \text{ independent set}}} |S|$$

- **Problem:** No continuity or gradients to exploit.
- **Brute-force search:** Find all $S \subset V$. Check if independent. Compute $|S|$. Complexity is $|\{S \subset V\}| = 2^n$.
- Can we do better than brute force search?

Motivating Example: Sequence Alignment

- A **DNA sequence** is a string in letters A, C, G, T. **e.g.** $X = AGGATC$.
- **Notation:** Will write $X = x_1 x_2 \ldots x_m$ where each $x_i \in \{A, C, G, T\}$.
- For any $i_1 < i_2 < \ldots < i_k$ the sequence $Z = x_{i_1} x_{i_2} \ldots x_{i_k}$ is a **subsequence** of X.

$X = ACCGTCGAGT$

$Z_1 = CTCA$ is a subsequence: $AC\,CG\,T\,C\,G\,AGT$

$Z_2 = ATGT$ is **not** a subsequence.

- Given two DNA sequences X and Y find the longest common subsequence.
- **Example:**

$$X = AC\,CG\,G\,TCGA\,G\,TGC\,G$$

$$Y = GT\,CG\,T\,TC\,G\,GATGC\,CC$$

$Z = CGTCGATGC$ is a common subsequence.

Motivating Example: Sequence Alignment

- Let $X = x_1 x_2 \dots x_M$ and $Y = y_1 y_2 \dots y_N$. Find longest common subsequence.
- Number of subsequences of X is 2^M. Bruteforce is clearly impractical.
- Define **prefix** $X_i = x_1 x_2 \dots x_i$ (sim. Y_j).
- Define $f(X_i, Y_j) =$ length of longest common subsequence of X_i and Y_j
- Note that $f(X_M, Y_N) = f(X, Y)$ is our goal.
- **Key Observation 1:**

$$f(X_i, Y_j) = \begin{cases} f(X_{i-1}, Y_{j-1}) + 1 & \text{if } x_i = y_j \\ \max\{f(X_i, Y_{j-1}), f(X_{i-1}, Y_j)\} & \text{otherwise} \end{cases}$$

- **Interpretation:** Can relate solution of problem to solutions of sub-problems *(optimal substructure)*.

Motivating Example: Sequence Alignment

- Know $f(X_4, Y_4) = \begin{cases} f(X_3, Y_3) + 1 & \text{if } x_4 = y_4 \\ \max\{f(X_4, Y_3), f(X_3, Y_4)\} & \text{otherwise} \end{cases}$
- So, to compute $f(X_4, Y_4)$ (may) need $f(X_4, Y_3)$.
- $f(X_4, Y_3) = \begin{cases} f(X_3, Y_2) + 1 & \text{if } x_4 = y_3 \\ \max\{f(X_4, Y_2), f(X_3, Y_3)\} & \text{otherwise} \end{cases}$
- **Key Observation 2:** Use sub-problems (*e.g.* computing $f(X_3, Y_3)$) multiple times. Makes sense to compute once and store.
- **Question:** How many sub-problems do we need to solve?

Motivating Example: Sequence Alignment

- Know $f(X_4, Y_4) = \begin{cases} f(X_3, Y_3) + 1 & \text{if } x_4 = y_4 \\ \max\{f(X_4, Y_3), f(X_3, Y_4)\} & \text{otherwise} \end{cases}$
- So, to compute $f(X_4, Y_4)$ (may) need $f(X_4, Y_3)$.
- $f(X_4, Y_3) = \begin{cases} f(X_3, Y_2) + 1 & \text{if } x_4 = y_3 \\ \max\{f(X_4, Y_2), f(X_3, Y_3)\} & \text{otherwise} \end{cases}$
- **Key Observation 2:** Use sub-problems (*e.g.* computing $f(X_3, Y_3)$) multiple times. Makes sense to compute once and store.
- **Question:** How many sub-problems do we need to solve?
- **Answer:** MN as need to compute $f(X_i, Y_j)$ once for all $i = 1, \ldots, M$ and $j = 1, \ldots, N$.
- An easy case: $f(X_1, Y_1) = \begin{cases} 1 & \text{if } x_1 = y_1 \\ 0 & \text{otherwise} \end{cases}$
- **Dynamic Programming Strategy:** Fill matrix A with $A_{ij} = f(X_i, Y_j)$ recursively. $A_{MN} = f(X_M, Y_N)$ is final answer.

Motivating Example: Sequence Alignment

- Take $X = GCCCTA$ and $Y = GCGT$

	G	C	C	C	T	A
G	1					
C						
G						
T						

$$
\begin{aligned}
f(X_2, Y_1) &= f(G, GC) \\
&= \begin{cases} f(X_1, Y_0) + 1 & \text{if } x_2 = y_1 \\ \max\{f(X_1, Y_1), f(X_2, Y_0)\} & \text{otherwise} \end{cases} \\
&= 1
\end{aligned}
$$

- Similarly $f(X_6, Y_1) = f(X_5, Y_1) = \ldots = f(X_1, Y_1) = 1$
- And: $f(X_1, Y_4) = f(X_1, Y_3) = \ldots = f(X_1, Y_1)$

Motivating Example: Sequence Alignment

- Cell (i, j) can be filled using data from cells $(i, j-1)$, $(i-1, j-1)$ or $(i-1, j)$. We put an arrow in cell (i, j) to indicate which of these is the case.

Motivating Example: Sequence Alignment

- Cell (i, j) can be filled using data from cells $(i, j-1)$, $(i-1, j-1)$ or $(i-1, j)$. We put an arrow in cell (i, j) to indicate which of these is the case.

	G	C	C	C	T	A
G	1	$\leftarrow 1$	$\leftarrow 1$	$\leftarrow 1$	$\leftarrow 1$	$\leftarrow 1$
C	$\uparrow 1$					
G	$\uparrow 1$					
T	$\uparrow 1$					

Motivating Example: Sequence Alignment

- Cell (i, j) can be filled using data from cells $(i, j-1)$, $(i-1, j-1)$ or $(i-1, j)$. We put an arrow in cell (i, j) to indicate which of these is the case.

	G	C	C	C	T	A
G	1	$\leftarrow 1$	$\leftarrow 1$	$\leftarrow 1$	$\leftarrow 1$	$\leftarrow 1$
C	$\uparrow 1$					
G	$\uparrow 1$					
T	$\uparrow 1$					

$$
\begin{aligned}
f(X_2, Y_2) &= f(GC, GC) \\
&= \begin{cases} f(X_1, Y_1) + 1 & \text{if } x_2 = y_2 \\ \max\{f(X_2, Y_1), f(X_1, Y_2)\} & \text{otherwise} \end{cases} \\
&= 2
\end{aligned}
$$

- Can check: $f(X_3, Y_2) = \ldots = f(X_6, Y_2) = 2$

Motivating Example: Sequence Alignment

	G	C	C	C	T	A
G	1	$\leftarrow 1$	$\leftarrow 1$	$\leftarrow 1$	$\leftarrow 1$	$\leftarrow 1$
C	$\uparrow 1$	$\nwarrow 2$	$\nwarrow 2$	$\nwarrow 2$	$\leftarrow 2$	$\leftarrow 2$
G						
T						

Motivating Example: Sequence Alignment

	G	C	C	C	T	A
G	1	$\leftarrow 1$	$\leftarrow 1$	$\leftarrow 1$	$\leftarrow 1$	$\leftarrow 1$
C	$\uparrow 1$	$\nwarrow 2$	$\nwarrow 2$	$\nwarrow 2$	$\leftarrow 2$	$\leftarrow 2$
G						
T						

	G	C	C	C	T	A
G	1	$\leftarrow 1$	$\leftarrow 1$	$\leftarrow 1$	$\leftarrow 1$	$\leftarrow 1$
C	$\uparrow 1$	$\nwarrow 2$	$\nwarrow 2$	$\nwarrow 2$	$\leftarrow 2$	$\leftarrow 2$
G	$\uparrow 1$	$\uparrow 2$	$\leftarrow\uparrow 2$	$\leftarrow\uparrow 2$	$\leftarrow\uparrow 2$	$\leftarrow\uparrow 2$
T						

- Note that if $f(i, j-1) = f(i-1, j)$ and $x_i \neq y_j$ then by definition:

$$f(i,j) = \max\{f(i,j-1), f(i-1,j)\} = f(i,j-1) \text{ or } f(i-1,j)$$

- In this case cell (i, j) can be filled using info from either cell $(i-1, j)$ or $(i, j-1)$. To represent this, we draw both arrows.

- completed table is shown below.

	G	C	C	C	T	A
G	1	$\leftarrow 1$	$\leftarrow 1$	$\leftarrow 1$	$\leftarrow 1$	$\leftarrow 1$
C	$\uparrow 1$	$\nwarrow 2$	$\nwarrow 2$	$\nwarrow 2$	$\leftarrow 2$	$\leftarrow 2$
G	$\uparrow 1$	$\uparrow 2$	$\leftarrow\uparrow 2$	$\leftarrow\uparrow 2$	$\leftarrow\uparrow 2$	$\leftarrow\uparrow 2$
T	$\uparrow 1$	$\uparrow 2$	$\leftarrow\uparrow 2$	$\leftarrow\uparrow 2$	$\nwarrow 3$	$\leftarrow 3$

Motivating Example: Sequence Alignment

- From Previous table, longest subsequence has length 3.
- **Question:** How to find the actual longest subsequence?

Motivating Example: Sequence Alignment

- From Previous table, longest subsequence has length 3.
- **Question:** How to find the actual longest subsequence?
- **Answer:** Traceback. Find a directed path from bottom right to top left.

Motivating Example: Sequence Alignment

- From Previous table, longest subsequence has length 3.
- **Question:** How to find the actual longest subsequence?
- **Answer:** Traceback. Find a directed path from bottom right to top left.

Motivating Example: Sequence Alignment

- From Previous table, longest subsequence has length 3.
- **Question:** How to find the actual longest subsequence?
- **Answer:** Traceback. Find a directed path from bottom right to top left.

	G	C	C	C	T	A
G	$\underline{1}$	$\leftarrow 1$	$\leftarrow 1$	$\leftarrow 1$	$\leftarrow 1$	$\leftarrow 1$
C	$\uparrow 1$	$\underline{\nwarrow 2}$	$\nwarrow 2$	$\nwarrow 2$	$\leftarrow 2$	$\leftarrow 2$
G	$\uparrow 1$	$\uparrow 2$	$\leftarrow\uparrow 2$	$\leftarrow\uparrow 2$	$\leftarrow\uparrow 2$	$\leftarrow\uparrow 2$
T	$\uparrow 1$	$\uparrow 2$	$\leftarrow\uparrow 2$	$\leftarrow\uparrow 2$	$\underline{\nwarrow 3}$	$\leftarrow 3$

- **Recall:** $f(X_i, Y_j) = \begin{cases} f(X_{i-1}, Y_{j-1}) + 1 & \text{if } x_i = y_j \\ \max\{f(X_i, Y_{j-1}), f(X_{i-1}, Y_j)\} & \text{otherwise} \end{cases}$
- A diagonal move up corresponds to a "hit" or addition to the subsequence.
- Underlined terms contribute to subsequence: GCT.
- Any directed path is equally valid. Will tell you different positioning of sequence CGT as a substring in X and Y.

Sequence Alignment: A larger example

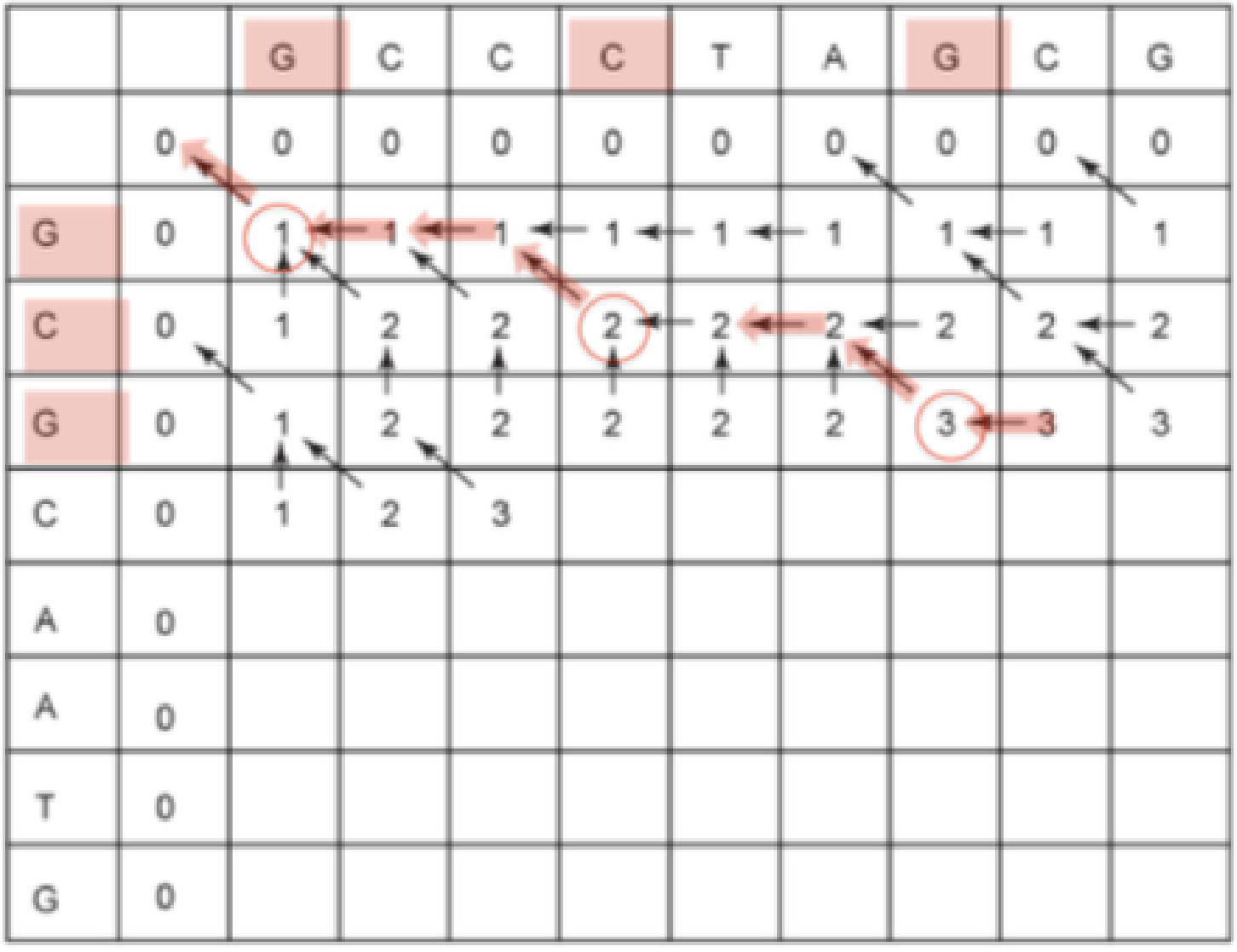


Figure: From: `http://cobweb.cs.uga.edu/~cai/courses/algo/2020Spring/`

Components of Dynamic Programming

Every Problem solvable by dynamic programming has the following:

- **Sub-problem structure:** Within problem of size n, call it $P_{(i,j)}$ can identify multiple problems of size $n-1$ or less.
- **Recursively definable objective function:** If $f(i,j)$ is optimal value for $P_{(i,j)}$ then:

$$f(i,j) = \text{ some function of } f(i-1,j), f(i,j-1), f(i-1,j-1), \ldots$$

- **Overlapping sub-problems** For different sub-problems of size n, say $P_{(i,j)}$ and $P_{(i',j')}$, computing $f(i,j)$ and $f(i^{'},j^{'})$ uses the same sub-sub-problem, say $P_{(i'',j'')}$.

Every dynamic programming algorithm has the same structure:

- Fill in table with the $f(i,j)$ in a **bottom-up** fashion.
- **Traceback** to find optimal solution.